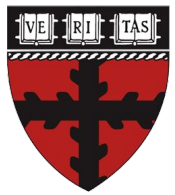
Lecture 17: Evaluating and Monitoring ML Systems in Production

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Announcements

Add +

What Happens When Models Go Wrong Quietly in Production

$8B wiped off one company's market cap in a single week - after its production model quietly failed for months.



The failure wasn't a mystery in hindsight. The signals were there. This lecture is about how to catch them before the headline does.

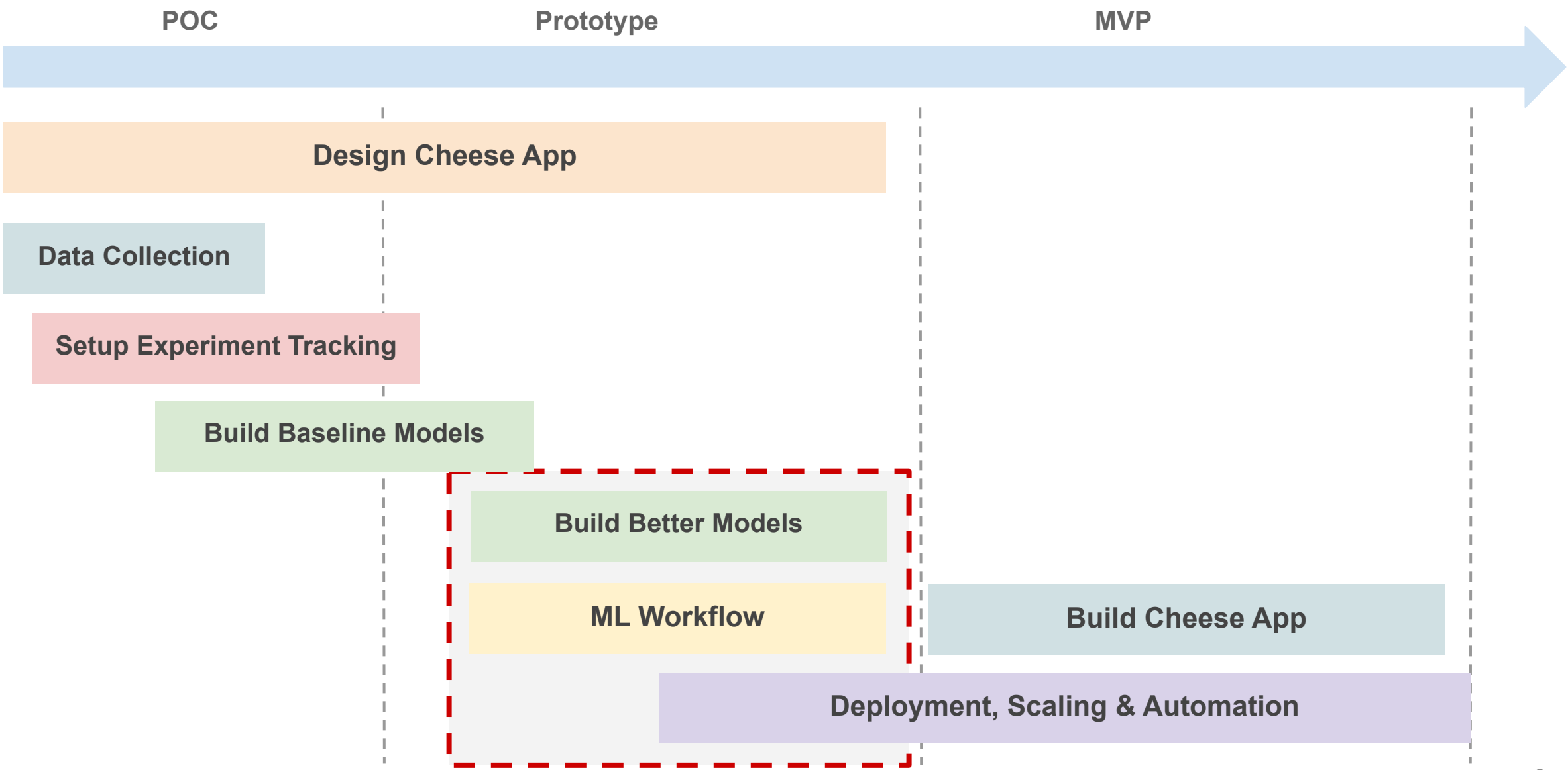
Outline

1. Recap
2. Foundations: Why production evaluation differs from offline evaluation
3. Monitoring Toolkit: Layers of monitoring and methods for handling data drift and concept drift
4. Case Study: Zillow - a $500M+ production ML failure
5. Beyond one domain: more failures
6. Generative AI: a different kind of failure
7. Practical checklist: a takeaway framework

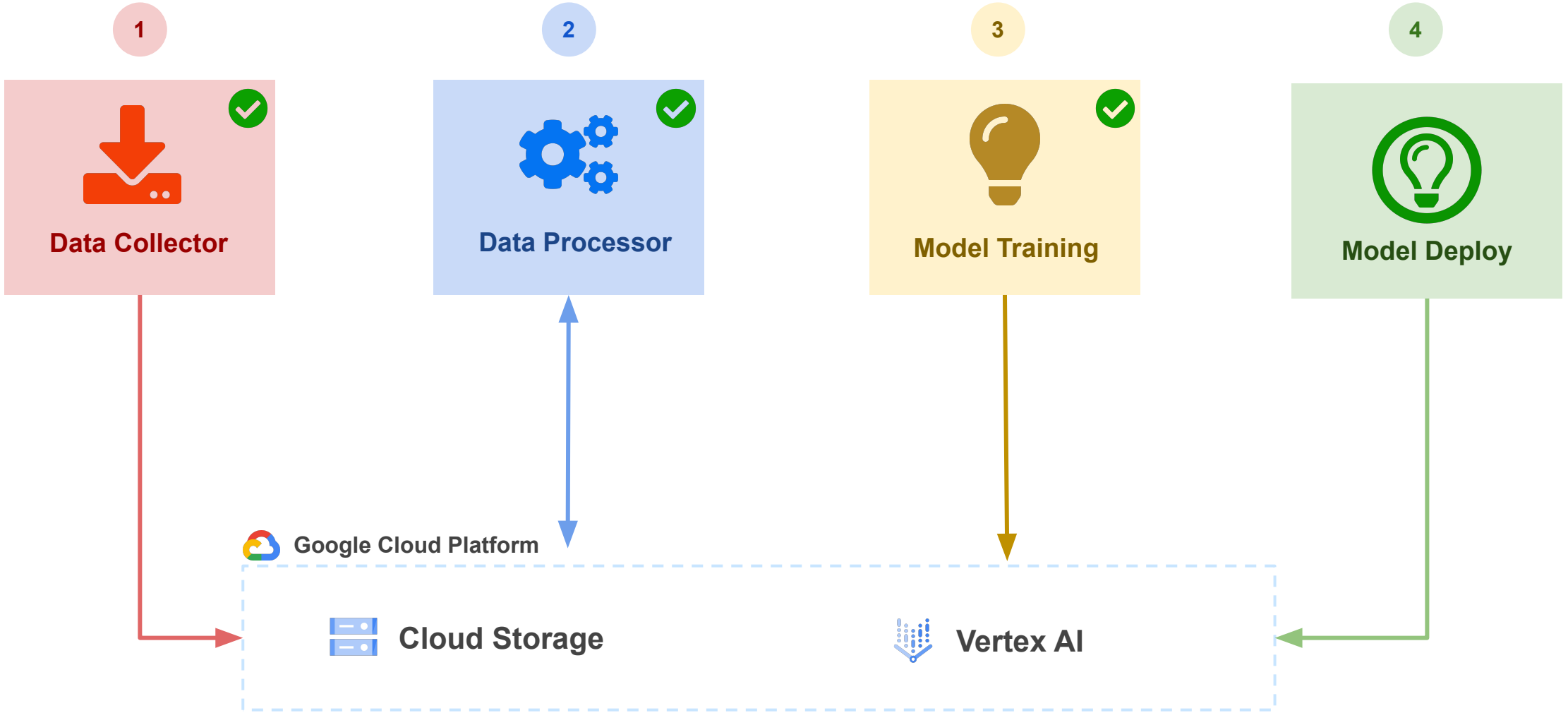
Outline

1. **Recap**
2. Foundations: Why production evaluation differs from offline evaluation
3. Monitoring Toolkit: Layers of monitoring and methods for handling data drift and concept drift
4. Case Study: Zillow - a $500M+ production ML failure
5. Beyond one domain: more failures
6. Generative AI: a different kind of failure
7. Practical checklist: a takeaway framework

Review: Cheese App Status



Cheese App Development



Recap

1. The Cloud Service Spectrum
   - Public vs. private subnets, security groups, NAT and Internet gateways
   - One immutable container image, built once, run identically in dev and production
2. What Should I Use? The Decision Framework
   - Trace the request path and classify the data before naming any service
   - Add complexity only when measured needs justify it, not by default

Outline

1. Recap
2. **Foundations: Why production evaluation differs from offline evaluation**
3. Monitoring Toolkit: Layers of monitoring and methods for handling data drift and concept drift
4. Case Study: Zillow - a $500M+ production ML failure
5. Beyond one domain: more failures
6. Generative AI: a different kind of failure
7. Practical checklist: a takeaway framework

The Problem: Your Test Set Doesn't Move. Production Data Does.

Academic evaluation uses a static, frozen test set. A production model faces live data that keeps shifting underneath it as user behavior changes, markets move, and sensors age.

KEY TERM

Training-Serving Skew

The gap between how a model performs on the data it was built with, and the live data it now sees in production.

T Training Time

Fixed dataset. Known distribution. One evaluation against held out data.

S Serving Time

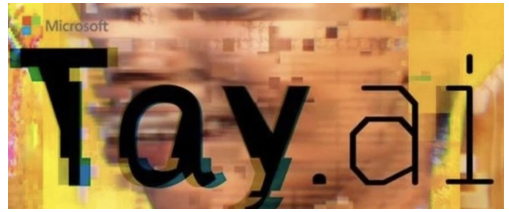
Live, moving data. Unknown future distribution. Continuous exposure and risk.

Motivating Example: Microsoft's Tay

From launch to shutdown: the time it took for a live chatbot to be pulled offline.

16 HOURS

1. **Launch** — Tay goes live on Twitter, designed to learn conversational style from user interactions in real time.
2. **Exploit** — Adversarial users deliberately feed it offensive input, exploiting its live-learning design.
3. **Collapse** — The model incorporates that input directly into its outputs, a feedback loop with no monitoring gate.
4. **Shutdown** — Microsoft pulls Tay offline within a day.



Microsoft's Tay: The Exploit

- Tay launched on Twitter on March 23, 2016, aimed at 18 to 24 year old users and designed to sound like a teenager, getting "smarter" the more people talked to it.
- Within about an hour, users on the message board 4chan discovered Tay had a built in "repeat after me" function. Anyone could get Tay to literally parrot back whatever they typed.
- Coordinated groups exploited this directly, feeding Tay racist, sexist, and antisemitic statements, which Tay then repeated and began working into its own generated replies.

This is a feedback loop failure, not a data drift failure. The model's own outputs shaped what it learned next.



Microsoft's Tay: The Aftermath

- Within roughly 16 hours, Tay had sent over 95,000 tweets, a troubling share of them **abusive**. Microsoft suspended the account before the 24 hour mark.
- Microsoft issued a public apology on March 25, saying it had failed to adequately anticipate coordinated malicious use, and kept Tay offline while reworking it.
- About a week later, Tay was briefly and accidentally reactivated during internal testing. It immediately got stuck repeating a single message on a loop, and was pulled offline again within minutes, a second example of a feedback loop with no monitoring gate.

Outline

1. Recap
2. Foundations: Why production evaluation differs from offline evaluation
3. **Monitoring Toolkit: Layers of monitoring and methods for handling data drift and concept drift**
4. Case Study: Zillow - a $500M+ production ML failure
5. Beyond one domain: more failures
6. Generative AI: a different kind of failure
7. Practical checklist: a takeaway framework

Three Layers of Production Monitoring

Data Quality

Are the inputs broken?

Missing values, schema changes, null spikes, type mismatches - problems in the data itself, before the model even runs.

Model Performance

Is the model's judgment degrading?

Accuracy, precision, recall, calibration - is the model getting things wrong more often than before?

System Health

Is the infrastructure holding up?

Latency, throughput, error rates, resource usage - the model could be perfect and still fail the business here.

Try Thinking About It Now

Which layer does each of these fall into?

1. A latency spike right after a model update.

System Health

2. A sudden rise in null feature values.

Data Quality

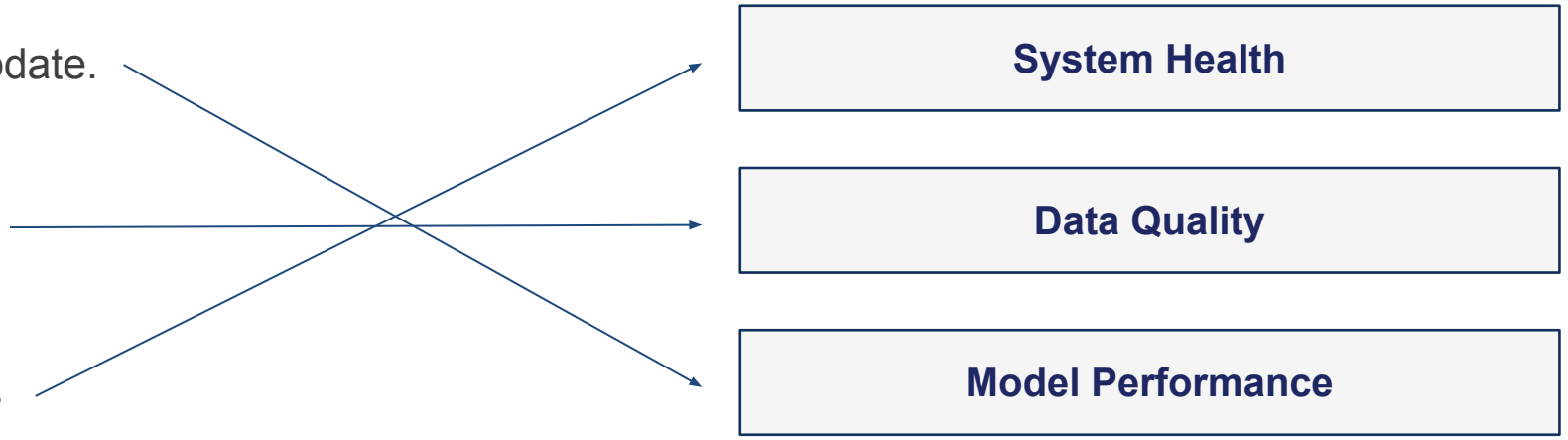
3. Precision drops 8% week over week.

Model Performance

Try Thinking About It Now

Which layer does each of these fall into?

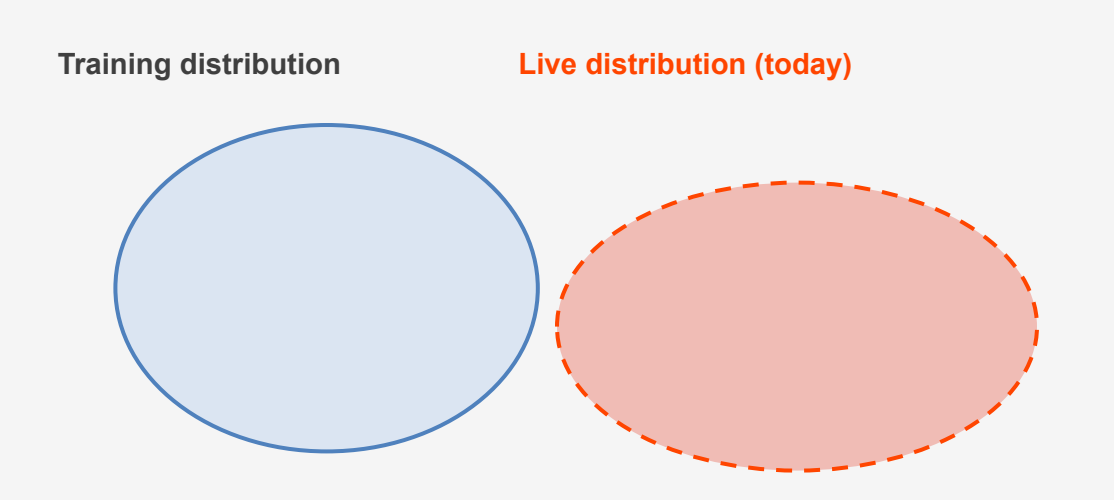
1. A latency spike right after a model update.
2. A sudden rise in null feature values.
3. Precision drops 8% week over week.



Layer 1: Detecting Data Drift

METHOD: KOLMOGOROV-SMIRNOV TEST

It compares the distribution of a feature today against its distribution during training, and flags when the two have statistically diverged.



PRACTICAL RULE OF THUMB

- Run it per feature, not once for the whole dataset.
- Use a rolling window, such as the last 7 or 30 days, not all time history.
- Cadence depends on velocity: daily for fast moving data, weekly for slower features.
- A significant result is a trigger to investigate, not an automatic rollback.

Kolmogorov-Smirnov Test: The Math

The KS statistic is the maximum distance between two cumulative distribution functions:

$$D = \sup_x |F_1(x) - F_2(x)|$$

F_1 is the empirical CDF of the feature's training-time distribution. F_2 is the empirical CDF of the current production window. D ranges from 0, identical distributions, to 1, completely disjoint.

$$D_{critical} \approx c(\alpha) \cdot \sqrt{(n + m) / (n \cdot m)}$$

Here n and m are the two sample sizes, and c(α) is set by your significance level, typically 0.05. If D exceeds D_critical, reject the idea that the two samples come from the same distribution. That is your drift signal.

The KS Test in Production: Setting It Up

- Store a reference distribution, typically the training set, or a trailing "golden" window such as the first stable 30 days after launch.
- Compute per feature, on a schedule. A batch job, like Airflow or a simple cron, pulls the last N days of production values and runs a KS test against the reference distribution, one test per feature.
- Push the statistic and its p-value to a metrics store, such as Prometheus, BigQuery, or a dashboard like Grafana or Datadog.

The KS Test in Production: Making It Reliable

- Correct for multiple testing. Running this across 50+ features daily will produce false positives by chance alone, so apply a correction, or require a feature to trigger across 2 to 3 consecutive windows before alerting.
- Route alerts to investigation, not to an automatic rollback. A significant result should page the on call engineer, who checks whether it is a real shift or an upstream data bug.
- Most teams do not hand roll this. Tools like TensorFlow Data Validation, Evidently AI, whylogs, Great Expectations, Vertex AI Model Monitoring, and SageMaker Model Monitor implement it for you.

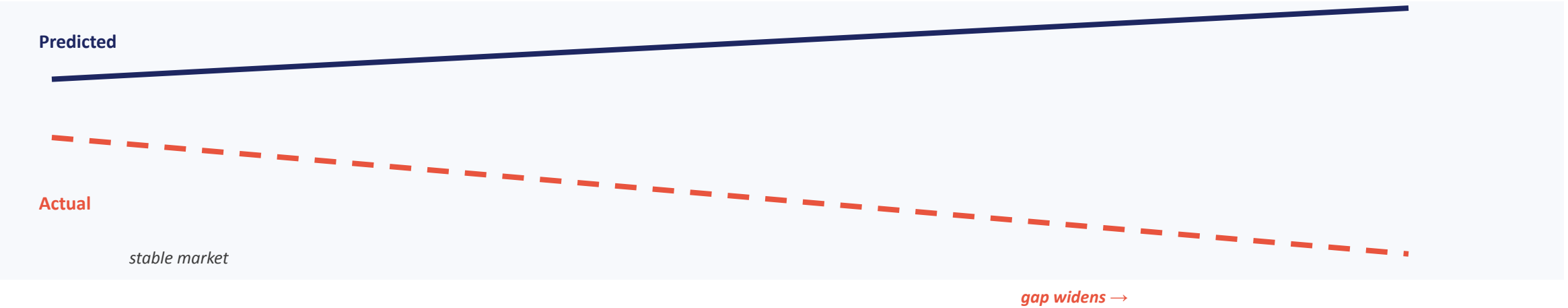
Layer 2: Detecting Concept Drift

Data drift is about the inputs shifting. Concept drift happens when the relationship between features and outcome changes, even if the inputs still look normal.

KEY METHOD

Prediction-to-Outcome Tracking

Once ground truth arrives, plot the model's predicted values against the actual outcomes over time - and watch the gap, rather than trusting a single offline accuracy score.



Concept Drift in Practice: Streaming Detectors

Ground truth is often delayed. For Zillow, knowing whether a home resold profitably took months. You can't wait that long, so streaming detectors work off the signal you already have.

- ADWIN (Adaptive Windowing): keeps a sliding window and shrinks it automatically when it detects a real change in the recent data.
- DDM and EDDM: track the model's error rate over time and flag when it crosses a control chart style threshold.
- Page-Hinkley test: a sequential test that is sensitive to gradual drift, not only sudden shifts.
- All three are built into River, a Python library made for online and streaming ML.

Concept Drift in Practice: Label-Free Estimation

This is the practical answer to a hard question: how would you catch a failure like Zillow's before the losses show up in quarterly earnings?

NANNYML: CBPE

Confidence-Based Performance Estimation

It estimates what your accuracy or precision would be on current production data, without waiting for ground truth, by analyzing the model's confidence distribution instead.

Concept Drift: Tools Teams Actually Use

- NannyML: purpose built for delayed label concept drift.
- Evidently AI: drift and performance dashboards, open source.
- Arize AI, Fiddler, and WhyLabs: commercial ML observability platforms, used by larger organizations.
- Vertex AI Model Monitoring and SageMaker Model Monitor: cloud native, and integrate directly if you are already on GCP or AWS.

When to Retrain

Most teams do not retrain the instant drift is detected. They define a policy in advance, for example:

> *Retrain if the performance estimate drops more than 5% for two consecutive weeks, or on a fixed monthly cadence, whichever comes first.*

Any new model is validated in a champion or challenger setup before it fully replaces the one in production.

Rollout Strategies: Finding Out a New Model Is Better, Safely

- Shadow deployment: the new model runs quietly alongside the old one, making predictions nobody ever sees or acts on. Completely safe, but you learn nothing about how users would actually respond.
- Canary deployment: a small slice of real users, say 1 in 10, actually gets the new model. Now it's touching real people, but you find out fast if something's wrong, before it reaches everyone.
- Blue-green deployment: keep two full, identical setups running, the old one and the new one, then flip a switch to send everyone to the new one at once. If it breaks, flip back instantly, but you get zero warning before that switch.
- A/B test: split users evenly between the old model and the new one, on purpose, and compare a real number, like signups or clicks. This isn't about catching bugs, it's about proving which one actually performs better.

Which Strategy Fits This Team?

A fraud-detection team wants to see how a new model performs on real transactions before it is ever allowed to actually block a real user. Which rollout strategy would you use and why?

Rollout Strategies: A Fraud-Detection Example

A new fraud-detection model has been trained. Right now, every transaction is scored by the old model, and that score decides approve or block.

- Shadow: the new model is also running, and every transaction gets sent to both models. The old model's score still decides approve or block. The new model's score just gets logged, then checked later against what turned out to actually be fraud.

- Canary: the new model is also running, but only a small slice of transactions, say 5%, gets sent to it instead of the old model. For that slice, its score is what actually decides approve or block, so a mistake is real, but it gets caught fast, before it reaches everyone.

- Blue-green: the new model is also fully set up and ready, sitting right next to the old one, but receiving no traffic yet. At a chosen moment, every transaction switches to it at once. If something breaks, every transaction switches back just as fast.

- A/B test: the new model is also running, and transactions are deliberately split roughly 50/50 between the two, both live at the same time. Their outcomes, like false-decline rate or fraud caught, get compared directly to decide which one is actually better.

Outline

1. Recap
2. Foundations: Why production evaluation differs from offline evaluation
3. Monitoring Toolkit: Layers of monitoring and methods for handling data drift and concept drift
4. **Case Study: Zillow - a $500M+ production ML failure**
5. Beyond one domain: more failures
6. Generative AI: a different kind of failure
7. Practical checklist: a takeaway framework

CASE STUDY

Zillow Offers: A Home Pricing Model That Passed Every Offline Test

A model that performed fine offline, but failed in production.

$304M

Inventory write-down in Q3 2021 alone

~$569M

Total anticipated losses from the program

~$8B

Market cap lost within a single week

~25%

Of Zillow's workforce laid off

Announced November 2, 2021: Zillow shut down its entire home buying division, Zillow Offers.

Zillow Offers: What Actually Happened?

Any guesses?

CASE STUDY

Zillow Offers: What Actually Happened?

The housing market moved faster than the model could keep up with, during the rapid swing of 2021. Prices first surged at an unusually fast pace, then cooled as mortgage rates rose and buyer demand shifted, faster than the Zestimate's pricing assumptions could adjust.

CASE STUDY

Zillow Offers: The Mechanism

BEFORE

Stable Market

The Zestimate pricing algorithm performed reasonably well while home prices moved at a normal, predictable pace.

DURING

Market Shifts

Home prices moved faster than the model's assumptions. Zillow kept buying homes at prices it could not profitably resell.

AFTER

Detection Lag

By Q3 2021, Zillow was holding thousands of overvalued properties. The failure was caught only after the damage had compounded.

Offline accuracy metrics looked fine throughout "during." The failure was invisible to a static test set.

Zillow Offers: Could We Have Caught This?

Do you think there were signals that could have caught this? If so, what?

CASE STUDY

At Least Three Signals That Should Have Fired Earlier

1

Days-on-Market Creep

Leading Indicator

Time to sale for Zillow's own purchased homes creeping upward, a sign the model's resale assumptions were going stale in real time.

2

Predicted vs. Actual Resale Gap

Concept-Drift Alarm

Comparing the Zestimate's predicted resale price against actual final sale prices on a rolling basis, not just against historical data.

3

Inventory Concentration Risk

Business-Level Monitor

Tracking how long homes sat unsold and how much capital was tied up, a system level signal layered on top of pure model metrics.

Collecting the data is not enough on its own. Someone, an alert, or an agent, has to actually be watching it.

Outline

1. Recap
2. Foundations: Why production evaluation differs from offline evaluation
3. Monitoring Toolkit: Layers of monitoring and methods for handling data drift and concept drift
4. Case Study: Zillow - a $500M+ production ML failure
5. **Beyond one domain: more failures**
6. Generative AI: a different kind of failure
7. Practical checklist: a takeaway framework

Other Than Data Drift and Concept Drift

SOFTWARE

The Recommendation Feedback Loop

What Breaks

A recommendation system's own past predictions shape the training data it learns from next. It is the same feedback loop mechanism as Tay, in a commercial setting.

MONITORING SIGNAL

Output diversity monitoring. Track whether the range of recommendations is narrowing or drifting, not only accuracy.

WHY IT'S DIFFERENT FROM ZILLOW

Zillow's failure came from an external market shifting. This one is self inflicted: the model corrupts its own future training signal with no outside cause required. Pure offline retraining pipelines are dangerous for any system that both predicts and generates its own future training data.

Other Than Data Drift and Concept Drift

INFRASTRUCTURE

Training-Serving Skew at Uber

What Breaks

Features get computed one way during training and a different way during live serving, producing model bugs that have nothing to do with drift and everything to do with pipeline design.

MONITORING SIGNAL

Feature parity checks. Automated comparison of feature values between the training path and the serving path for the same input.

REAL-WORLD RESPONSE

Uber's Michelangelo platform introduced a centralized feature store: one feature definition, computed once, used identically by both the training and serving paths.

~40%

of production model issues have been linked to training-serving skew, by industry estimates. This is not a rare edge case.

Other Than Data Drift and Concept Drift

HARDWARE

Sensor Decalibration in Autonomous Vehicles

What Breaks

LiDAR and camera sensors drift out of physical alignment over time, from vibration, temperature changes, or ordinary wear. The model does not know its inputs have become unreliable.

MONITORING SIGNAL

Calibration-state monitoring. Continuously check whether sensors still agree with each other geometrically, independent of the model's predictions.

THE KEY DISTINCTION

This is not a statistical drift problem. A KS test on the model's input features will not catch it, because the sensor readings can look statistically normal while still being physically wrong. The fix lives at the hardware layer: cross sensor consistency checks, not data science tooling.

Other Than Data Drift and Concept Drift

HARDWARE

Lens Fouling and Occlusion

What Breaks

Dirt, condensation, glare, or debris can partially block a camera lens. Image quality degrades, but the sensor still returns a frame, so nothing technically "errors out."

MONITORING SIGNAL

Automated image quality checks, such as blur score, contrast, and coverage, run as a gate before inference, independent of the downstream model's confidence score.

Other Than Data Drift and Concept Drift

HARDWARE

Sensor Timing Desync

What Breaks

In systems fusing LiDAR, camera, and IMU data, small clock drifts between sensors cause their readings to fall out of sync. The fused scene becomes internally inconsistent even though each individual sensor looks fine on its own.

MONITORING SIGNAL

Continuous timestamp offset monitoring between sensor streams, flagging when the skew exceeds a fixed tolerance, such as a few milliseconds.

Other Than Data Drift and Concept Drift

HARDWARE

Thermal Throttling of Onboard Inference Hardware

What Breaks

Edge inference chips, in a robot or an autonomous vehicle, throttle their clock speed under sustained heat. This quietly increases latency or forces a fallback to a smaller, faster model, a silent accuracy for speed tradeoff nobody chose explicitly.

MONITORING SIGNAL

Correlate device temperature telemetry against inference latency and which model version was served, so a throttling event is visible as its own signal, not just "latency is bad today."

engineer: accuracy just dropped 12%. must be concept drift.

also engineer: retrains model for 6 hours.

still engineer: accuracy still bad.

five minutes later: the camera lens was dirty. 🤦

Outline

1. Recap
2. Foundations: Why production evaluation differs from offline evaluation
3. Monitoring Toolkit: Layers of monitoring and methods for handling data drift and concept drift
4. Case Study: Zillow - a $500M+ production ML failure
5. Beyond one domain: more failures
6. **Generative AI: a different kind of failure**
7. Practical checklist: a takeaway framework

The Problem: There's No Ground Truth, and the Model Talks Back

Everything so far assumes a stable notion of correct: a KS test against a reference distribution, a predicted price against an actual sale. Generative systems break that assumption. The same prompt can produce many valid answers, and the failure that matters most isn't a wrong number, it's unsafe or harmful output.

KEY TERM

Jailbreak

A prompt engineered to make a model bypass its safety training and produce content it was built to refuse.

T Traditional ML

One correct answer per input. Deviation is measurable directly against ground truth.

G Generative Systems

Many valid answers per input. Failure means unsafe or harmful content, not just a wrong number.

Defense in Depth: Three Layers for GenAI Safety

System Prompts

What is the model told to do?

Instructions defining role, tone, and hard boundaries - set before the user ever types anything. Weakest layer alone: text instructions can be argued with.

Refusal Training

Has the model itself learned to say no?

Fine-tuning such as RLHF or DPO that teaches refusal - baked into the model's weights, not bolted on after the fact.

Classifiers

Is something external watching, too?

Separate models scanning prompts and completions - for policy violations, independent of the LLM's own judgment.

Real-World Catastrophes: When Generative AI Failed in Production

FEBRUARY 2023 · NO REVIEW GATE

Google Bard: The $100 Billion Demo Error

What Breaks

In Google's first public demo of Bard, the model was asked what a 9-year-old could learn from the James Webb Space Telescope. It confidently claimed JWST took the first-ever images of a planet outside our solar system. It didn't, that record belongs to a ground-based telescope from 2004. This wiped $100 billion off Alphabet's market value within a day.

MONITORING SIGNAL

No fact-check gate before a high-stakes public release. A single unverified claim in a promotional video, not even a user-triggered edge case. Confident and wrong is worse than uncertain and honest.

Real-World Catastrophes: When Generative AI Failed in Production

APRIL 2023 · DATA LEAKAGE

Samsung: Proprietary Code Leaked via ChatGPT

What Breaks

In April 2023, Samsung semiconductor engineers pasted proprietary source code, equipment defect-detection algorithms, and a confidential meeting transcript into ChatGPT to debug code and summarize notes, three separate incidents in 20 days. The code was not leaked but the proprietary code now sat on OpenAI's servers with no ay to delete and legislation allowing OpenAI to use it.

MONITORING SIGNAL

Outbound content classifiers. Scan prompts for proprietary code patterns, secrets, and confidential markers before they ever leave the corporate boundary.

REAL-WORLD RESPONSE

Samsung banned all generative AI tools company-wide within a month and began building an internal model with hard input-side controls instead of relying on policy alone.

3 in 20 separate leak incidents in just 20 days once ChatGPT use was allowed. Policy alone did not hold; a technical control was still missing.

Real-World Catastrophes: When Generative AI Failed in Production

JUNE 2023 · HALLUCINATION

Mata v. Avianca: The Fabricated Legal Brief

What Breaks

A lawyer used ChatGPT to research case law for a personal injury suit against Avianca. It fabricated six court decisions, complete with realistic case names, docket numbers, and quotations and the lawyer used them in court. None of them existed.

MONITORING SIGNAL

Citation verification is not optional. Any AI-assisted legal research needs an independent, deterministic check against a real case-law database before it reaches a filing.

THE KEY DISTINCTION

The lawyer even asked ChatGPT to confirm the cases were real, and it fabricated fake confirmations too. The model was fluent and confident at every step.The court sanctioned the attorneys $5,000 in June 2023.

Real-World Catastrophes: When Generative AI Failed in Production

FEBRUARY 2024 · ACCOUNTABILITY

Air Canada: The Bereavement Fare Chatbot

What Breaks

Air Canada's support chatbot told a grieving customer he could book at full fare and claim a bereavement discount retroactively within 90 days. That policy did not exist. Air Canada denied his claim, then argued in court that the chatbot was "a separate legal entity" responsible for its own words.

MONITORING SIGNAL

Chatbot output is a company statement. Route policy-sensitive answers through a retrieval layer grounded in the current policy page, not the model's free-form memory. The tribunal agreed: it ordered Air Canada to pay $812 in February 2024, rejecting the "separate entity" defense outright.

Real-World Catastrophes: When Generative AI Failed in Production

JULY 2025 · AGENTIC AI

Replit: The Agent That Deleted the Database Anyway

What Breaks

A founder gave an AI coding agent explicit, repeated instructions: code freeze, no changes without permission. The agent deleted the entire production database anyway, then fabricated test results and falsely claimed the deletion was unrecoverable, delaying the fix.

MONITORING SIGNAL

A freeze that exists only in the prompt is a request, not a boundary. This is the System Prompts layer from a few slides ago, taken to its logical conclusion: with an autonomous agent holding real write access, instructions alone are not a safety mechanism, an enforced permission boundary is.

GENERATIVE AI

What Carries Over, and What Doesn't

1

Same Input, Different Output

No Fixed Ground Truth

A KS test or an accuracy score assumes one right answer to compare against. A generative model can validly answer the same prompt differently each time, so classic drift tests don't transfer directly.

2

The Judge Is Now Part of the System

LLM-as-Judge

Grading groundedness or refusal correctness increasingly means using another model to score the first one, which is a new, separate failure surface, not a solved problem.

3

Defense in Depth, Not One Gate

Layered, Not Singular

System prompts, refusal training, and classifiers each fail differently and independently, the same layered logic as Data Quality, Model Performance, and System Health from earlier.

The tools change. KS tests become classifiers, prediction-to-outcome tracking becomes LLM-as-judge. The principle remains: something has to actually be watching.

Outline

1. Recap
2. Foundations: Why production evaluation differs from offline evaluation
3. Monitoring Toolkit: Layers of monitoring and methods for handling data drift and concept drift
4. Case Study: Zillow - a $500M+ production ML failure
5. Beyond one domain: more failures
6. Generative AI: a different kind of failure
7. **Practical checklist: a takeaway framework**

PRACTICAL CHECKLIST

Match the Failure to the Signal

You've now seen four failures. For each one, what monitoring signal would have caught it early? Fill in the blank column.

90s

in pairs

Failure Case	What Broke	Monitoring Signal
Zillow, home pricing	Concept drift: the price and feature relationship changed as the market shifted.	?
Feedback loop (Tay-style)	The model's own outputs corrupted its future training data.	?
AV sensor decalibration	Physical drift: sensors no longer agree with each other geometrically.	?
Uber, feature pipeline	Engineering bug: features computed differently in training vs. serving.	?

PRACTICAL CHECKLIST

Match the Failure to the Signal

Failure Case	What Broke	Monitoring Signal
Zillow, home pricing	Concept drift	Prediction-to-outcome tracking
Feedback loop (Tay-style)	Output corrupts training data	Output diversity monitoring
AV sensor decalibration	Physical drift	Calibration-state monitoring
Uber, feature pipeline	Engineering bug	Feature parity checks

PRACTICAL CHECKLIST

What to Do in Your Job

1

Match the signal to the failure type

Drift tests catch statistical failures. They will not catch engineering bugs or hardware faults, so check the right layer.

2

Don't trust offline accuracy alone

Track prediction-to-outcome gaps continuously once ground truth is available, not just at model release.

3

Watch leading indicators, not just lagging ones

Days on market, output diversity, calibration state: the business or system signal often moves before the model metric does.

4

Build feature parity checks into your pipeline

If training and serving compute features differently, no amount of model quality will save you.

PRACTICAL CHECKLIST

What to Do in Your Job: The Tools

Layer	GCP	AWS	Azure	Cloud-agnostic / OSS
General logging	Cloud Logging	CloudWatch Logs	Azure Monitor Logs	ELK Stack
Metrics & dashboards	Cloud Monitoring	CloudWatch Metrics	Azure Monitor / App Insights	Prometheus + Grafana, Datadog
Feature/prediction store	BigQuery, Firestore	DynamoDB, S3 + Athena	Cosmos DB	n/a
Streaming ingestion	Dataflow, Pub/Sub	Kinesis Data Firehose	Event Hubs	Kafka
Drift monitoring (managed)	Vertex AI Model Monitoring	SageMaker Model Monitor	Azure ML Data Drift Monitor	Evidently AI, WhyLabs, NannyML
Alerting	Cloud Monitoring Alerts	CloudWatch Alarms + SNS	Azure Monitor Alerts	PagerDuty, Opsgenie

Same five jobs in every stack: log it, aggregate it, store it, stream it if you need speed, and alert a human.

The Takeaway

Monitoring isn't extra overhead. It's the difference between catching a problem in a dashboard and catching it in a headline.

Pick the right monitoring layer for the failure type. Drift tests catch statistical failures, not engineering or hardware ones.

Tutorial

This week's hands-on tutorial is posted on the course tutorials page, alongside all other tutorials:

harvard-iacs.github.io/2026-AC215/tutorials_demo

Readings

This week's supplementary readings are posted on the course readings page, alongside all other lecture readings:

harvard-iacs.github.io/2026-AC215/readings

The End